Michael Huang
EN.625.633.81
Homework 6

1.

Example 2.7. Example 2.2 did not provide a general algorithm to simulate beta $\mathcal{B}e(\alpha, \beta)$ random variables. We can, however, construct a toy algorithm based on the Accept–Reject method, using as the instrumental distribution the uniform $\mathcal{U}_{[0,1]}$ distribution when both α and β are larger than 1. (The generic `rbeta` function does not impose this restriction.)

The upper bound M is then the maximum of the beta density, obtained for instance by `optimize` (or its alias `optimise`):

```
> optimize(f=function(x){dbeta(x,2.7,6.3)},
+ interval=c(0,1),max=T)$objective
[1] 2.669744
```

Since the candidate density g is equal to one, the proposed value Y is accepted if $M \times U < f(Y)$, that is, if $M \times U$ is under the beta density f at that realization. Note that generating $U \sim \mathcal{U}_{[0,1]}$ and multiplying by M is equivalent to generating $U \sim \mathcal{U}_{[0,M]}$. For $\alpha = 2.7$ and $\beta = 6.3$, an alternative `R` implementation of the Accept–Reject algorithm is

```
> Nsim=2500
> a=2.7;b=6.3
> M=2.67
> u=runif(Nsim,max=M)     #uniform over (0,M)
> y=runif(Nsim)           #generation from g
> x=y[u<dbeta(y,a,b)]     #accepted subsample
```

and the left panel in Figure 2.4 shows the results of generating 2500 pairs (Y, U) from $\mathcal{U}_{[0,1]} \times \mathcal{U}_{[0,M]}$. The black dots $(Y, Ug(Y))$ that fall under the density f are those for which we accept $X = Y$, and we reject the grey dots $(Y, Ug(Y))$ that fall outside. It is again clear from this graphical representation that the black dots are uniformly distributed over the area under the density f. Since the probability of acceptance of a given simulation is $1/M$ (Exercise 2.5), with $M = 2.67$ we accept approximately $1/2.67 = 37\%$ of the values. ◀

Exercise 6.7 Referring to Example 2.7, consider a $\mathcal{B}e(2.7, 6.3)$ target density.

a. Generate Metropolis–Hastings samples from this density using a range of independent beta candidates from a $\mathcal{B}e(1, 1)$ to a beta distribution with small variance. (*Note:* Recall that the variance is $ab/(a+b)^2(a+b+1)$.) Compare the acceptance rates of the algorithms.

b. Suppose that we want to generate a *truncated* beta $\mathcal{B}e(2.7, 6.3)$ restricted to the interval (c, d) with $c, d \in (0, 1)$. Compare the performance of a Metropolis–Hastings algorithm based on a $\mathcal{B}e(2, 6)$ proposal with one based on a $\mathcal{U}(c, d)$ proposal. Take $c = .1, .25$ and $d = .9, .75$.

We are generating based on a beta $Be(2.7, 6.3)$ target density.

(a)

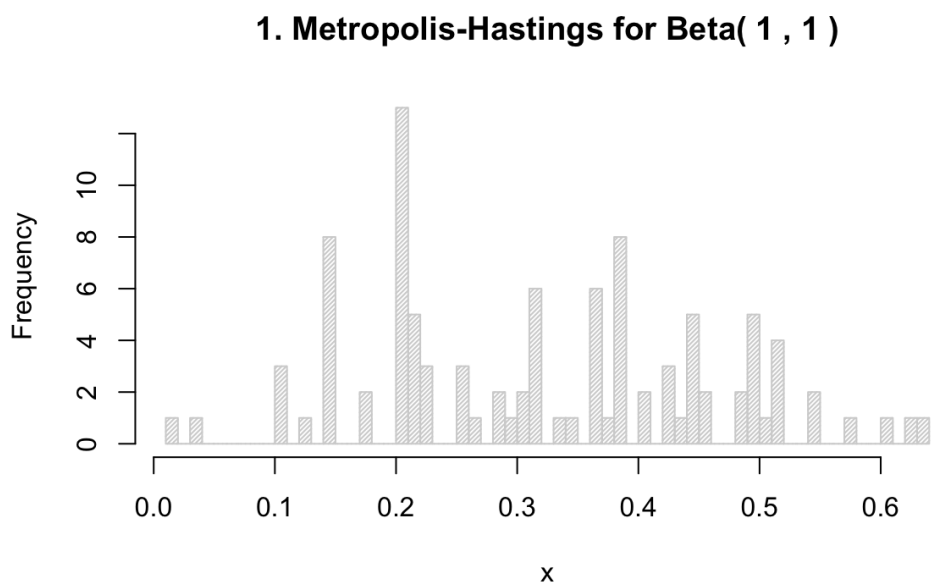
Using the problem statement, we know that the target distributions' variance is $\frac{2.7 \cdot 6.3}{(2.7+6.3)^2 \cdot (2.7+6.3+1)} = \frac{17.01}{810} = 0.021$. We'll test using the following beta candidates:
- $Be(1, 1)$: given, variance $= \frac{1 \cdot 1}{(1+1)^2 \cdot (1+1+1)} = \frac{1}{12} = 0.08333333333$
- $Be(2, 2)$: given, variance $= \frac{2 \cdot 2}{(2+2)^2 \cdot (2+2+1)} = \frac{4}{80} = \frac{1}{20} = 0.05$, lower than $Be(1, 1)$
- $Be(2, 4)$: given, variance $= \frac{2 \cdot 4}{(2+4)^2 \cdot (2+4+1)} = \frac{8}{252} = 0.03174603174$, even lower
- $Be(2, 5)$: given, variance $= \frac{2 \cdot 5}{(2+5)^2 \cdot (2+5+1)} = \frac{10}{392} = 0.02551020408$, even lower
- $Be(3, 6)$: given, variance $= \frac{3 \cdot 6}{(3+6)^2 \cdot (3+6+1)} = \frac{18}{810} = 0.02222222222$, even lower

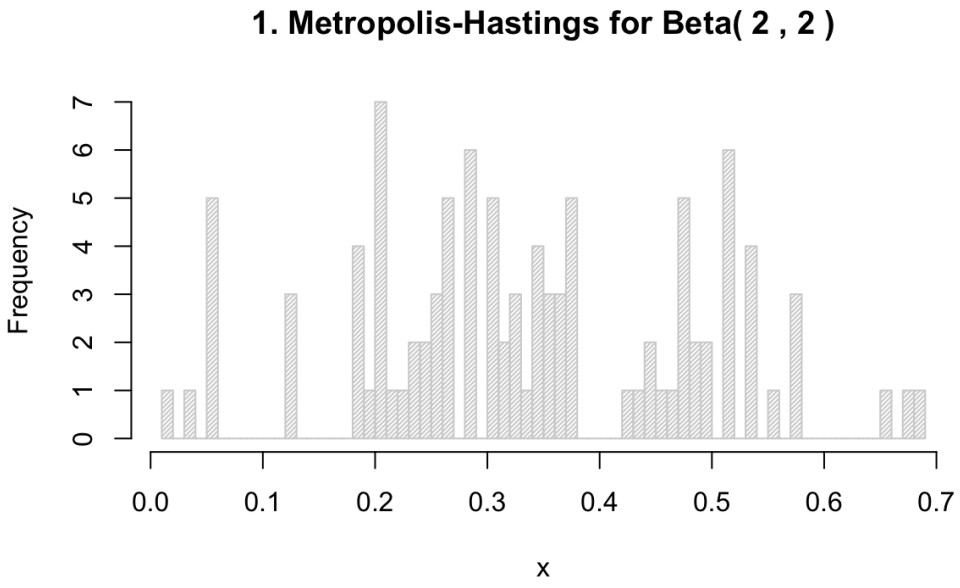
```
> met_hastings_beta = function(n, prop_alpha, prop_beta) {
+
+   x = rep(0, n)
+   j = 1
+   acc = 0
+   # Due to the nature of the beta distribution
+   x[1] = 0.5
+
+   while (j < n) {
+
```

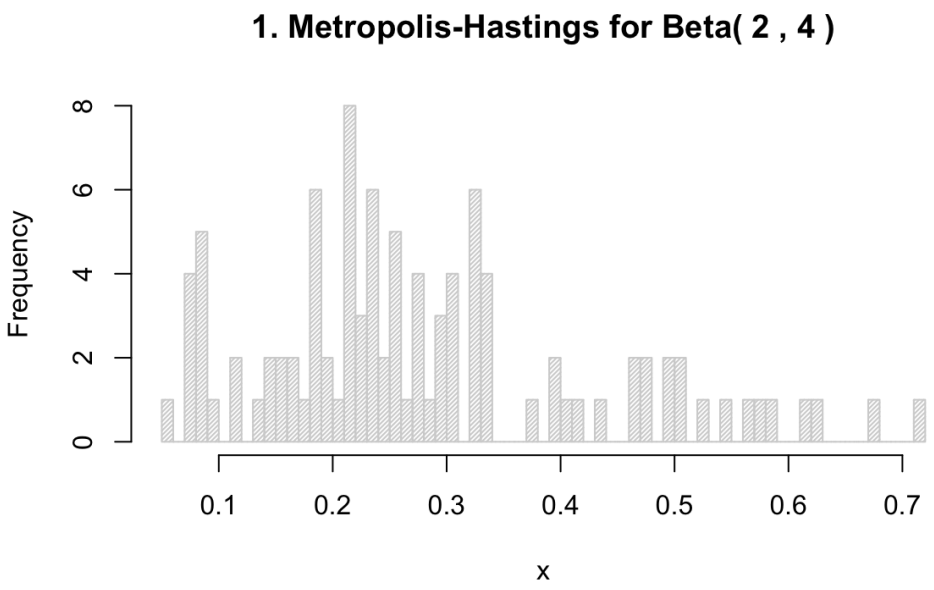
```
+     # Sample candidate value
+     x_cand = rbeta(1, prop_alpha, prop_beta)
+
+     # Calculate acceptance probability
+     acc_prob = acc_prob_calc_beta(x_cand, x[j], prop_alpha, prop_beta)
+
+     # Decide whether to accept or not
+     acc_or_not = rbinom(1, 1, acc_prob)
+
+     # if accept or not
+     if (acc_or_not == 1) {
+       acc = acc + 1
+       j = j + 1
+       x[j] = x_cand
+     }
+     else {
+       j = j + 1
+       x[j] = x[j-1]
+     }
+   }
+
+   acc_probability = acc/n
+   print(paste("Acceptance probability:", acc_probability))
+   hist(x, breaks = 50, density = 50, main = paste(
+     "1. Metropolis-Hastings for Beta(", prop_alpha, ",", prop_beta, ")")
+   );
+ }
>
> acc_prob_calc_beta = function(candidate, former, prop_alpha, prop_beta)
{
```

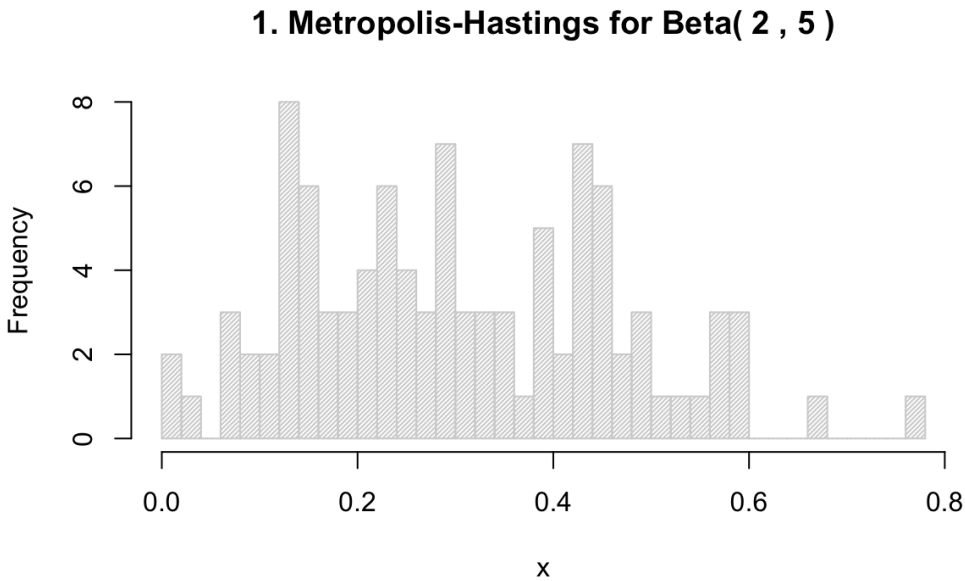
```
+
+   if (candidate <= 0 || candidate >= 1) {
+     acc_probability = 0
+   }
+   else {
+     # Target densities
+     dens_candidate = dbeta(candidate, 2.7, 6.3)
+     dens_former = dbeta(former, 2.7, 6.3)
+
+     # Proposal densities
+     g_former_given_candidate = dbeta(former, prop_alpha, prop_beta)
+     g_candidate_given_former = dbeta(candidate, prop_alpha, prop_beta)
+
+     acc_probability = (dens_candidate * g_former_given_candidate) / (dens_
* g_candidate_given_former);
+   }
+
+   acc_probability = min(1, acc_probability);
+   return(acc_probability)
+ }
>
> # 1. (a)
>
> # Using Beta(1, 1)
> met_hastings_beta(100, 1, 1)
[1] "Acceptance probability: 0.44"
>
> # Using Beta(2, 2)
> met_hastings_beta(100, 2, 2)
[1] "Acceptance probability: 0.57"
```

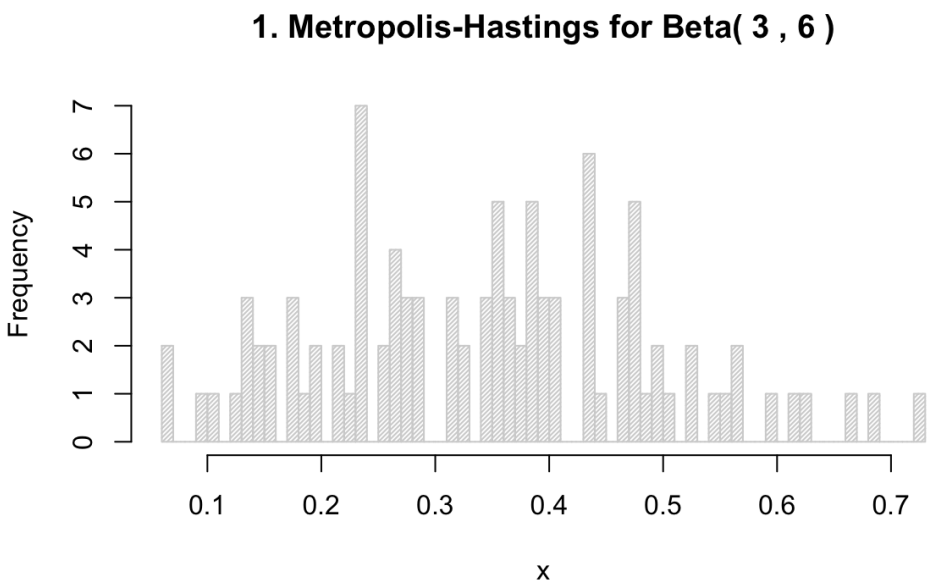
```
>
> # Using Beta(2, 4)
> met_hastings_beta(100, 2, 4)
[1] "Acceptance probability: 0.85"
>
> # Using Beta(2, 5)
> met_hastings_beta(100, 2, 5)
[1] "Acceptance probability: 0.91"
>
> # Using Beta(3, 6)
> met_hastings_beta(100, 3, 6)
[1] "Acceptance probability: 0.87"
```




1. Metropolis-Hastings for Beta(2 , 2)
Frequency
0
1
2
3
4
5
6
7
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
x


1. Metropolis-Hastings for Beta(2 , 4)
Frequency
0
2
4
6
8
0.1
0.2
0.3
0.4
0.5
0.6
0.7
x





Compared with the accept/reject method, the acceptance rate of the Metropolis-Hastings algorithm is consistently higher (compared with the stated acceptance rate of 37% of values in the example mentioned in the exercise). In addition, we see the progression of increased acceptance rate as expected here, with higher acceptance rates as we get lower variances for proposed distributions, with the sole exception of Beta(3, 6). Looking at a graph, it seems that this distribution is more right-skewed and has higher concentrations of densities on the left than the target or Beta(2, 5)

which seems to hit this slightly better.

(b)

To generate a truncated distribution, we intuitively need to adjust the distribution in a way such that the entire probability space is covered within (c, d). Given a pdf $f(x)$, we have $\int_0^1 f(x)dx = 1$ which captures the entire probability space, but now we must scale such that $\int_c^d f(x)dx = 1$. To do so, we would need to do so by multiplying by that exact integral, i.e. we would adjust such that

$$f_{truncated}(x) = \frac{f(x)}{\int_c^d f(x)dx}$$

which works as

$$\int_c^d f_{truncated}(x)dx = \int_c^d \frac{f(x)}{\int_c^d f(x)dx}dx = \frac{1}{\int_c^d f(x)dx}\int_c^d f(x)dx = \frac{\int_c^d f(x)dx}{\int_c^d f(x)dx} = 1$$

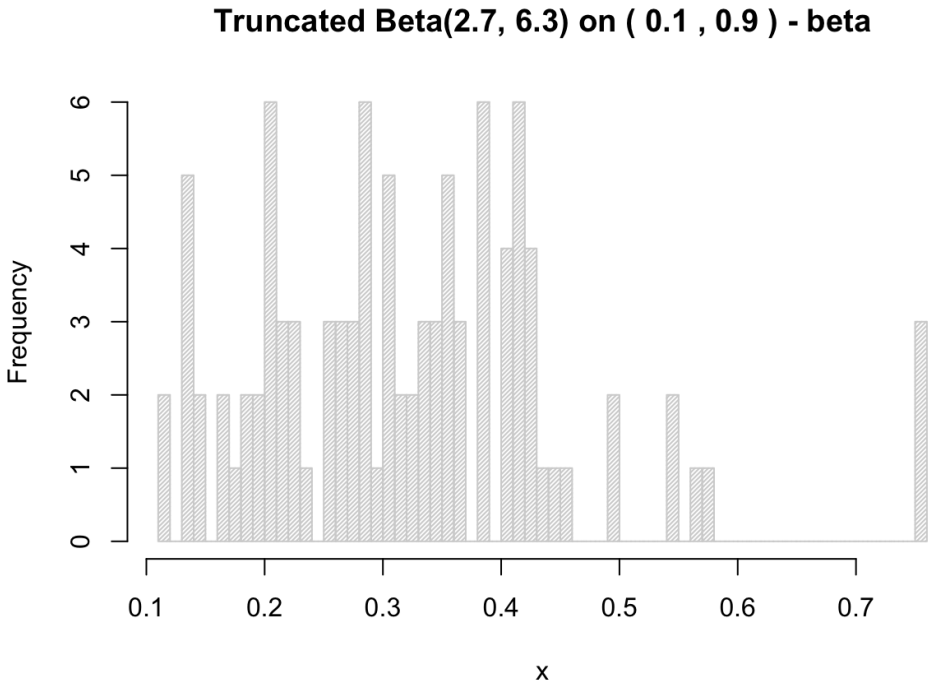
So we adjust the code accordingly:

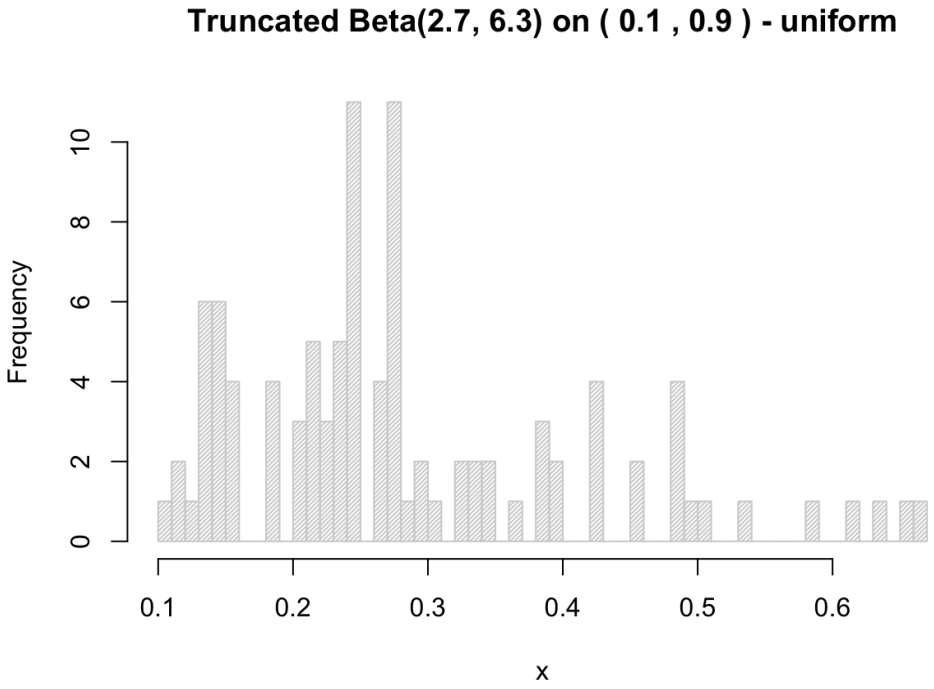
```
> met_hastings_truncated_beta = function(n, c, d, prop_function = "beta")
{
+
+   x = rep(0, n)
+   j = 1
+   acc = 0
+   # Start in middle of truncation interval
+   x[1] = (c + d)/2
+
+   # We use Beta(2,6)
+   prop_alpha = 2
+   prop_beta = 6
+
+   while (j < n) {
+
+     # Sample candidate value, based on proposal function
```

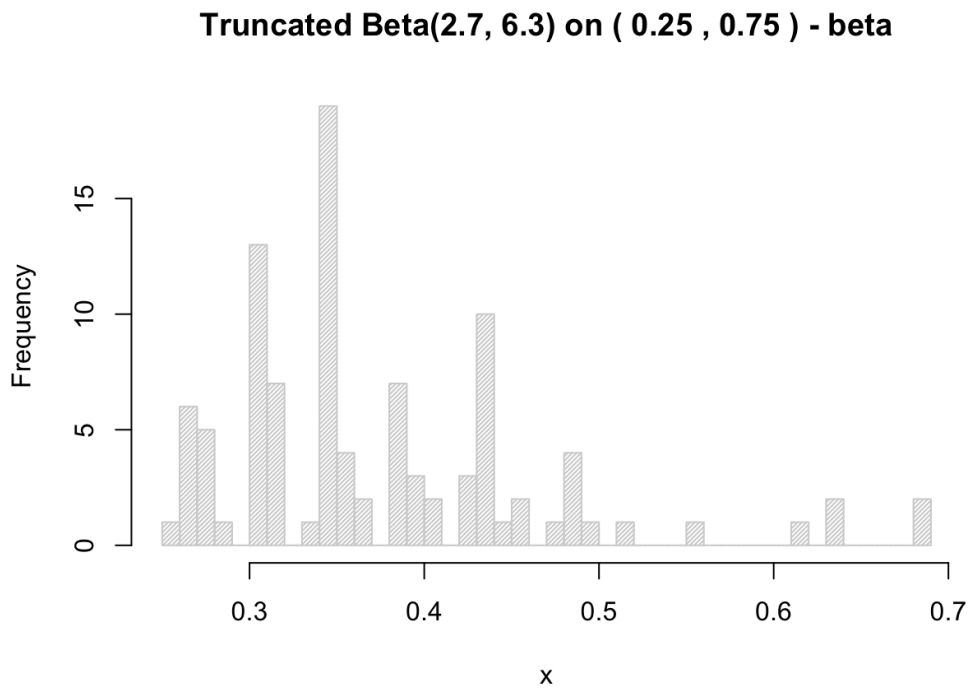
```
+     if (prop_function == "beta") {
+       x_cand = rbeta(1, prop_alpha, prop_beta)
+     } else if (prop_function == "uniform") {
+       x_cand = runif(1, c, d)
+     }
+
+     # Calculate acceptance probability
+     acc_prob = acc_prob_calc_truncated(x_cand, x[j], c, d,
+                                        prop_function, prop_alpha, prop_beta
+
+     # Decide whether to accept or not
+     acc_or_not = rbinom(1, 1, acc_prob)
+
+     # if accept or not
+     if (acc_or_not == 1) {
+       acc = acc + 1
+       j = j + 1
+       x[j] = x_cand
+     }
+     else {
+       j = j + 1
+       x[j] = x[j-1]
+     }
+   }
+
+   acc_probability = acc/n
+   print(paste("Interval: (", c, ",", d, ")"))
+   print(paste("Proposal function:", prop_function))
+   print(paste("Acceptance probability:", acc_probability))
+
```

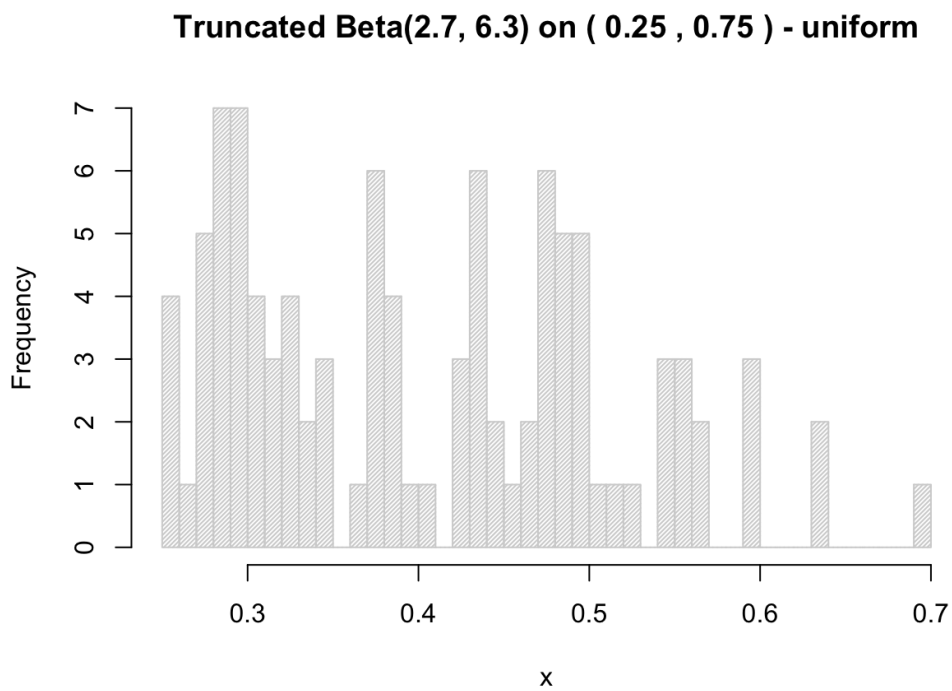
```
+   hist(x, breaks = 50, density = 50,
+        main = paste("Truncated Beta(2.7, 6.3) on (", c, ",", d,  ") -",
prop_function))
+ }
>
>
> acc_prob_calc_truncated = function(candidate, former, c, d,
+                                    prop_function, prop_alpha, prop_beta)
{
+
+   # Reject if candidate is outside truncated interval
+   if (candidate <= c || candidate >= d) {
+     return(0)
+   }
+
+   # Truncated target densities, adjusted by integral
+   dens_candidate = (dbeta(candidate, 2.7, 6.3) /
+     (pbeta(d, 2.7, 6.3) - pbeta(c, 2.7, 6.3)))
+   dens_former = (dbeta(former, 2.7, 6.3) /
+     (pbeta(d, 2.7, 6.3) - pbeta(c, 2.7, 6.3)))
+
+   # Proposal densities
+   if (prop_function == "beta") {
+     g_former_given_candidate = dbeta(former, prop_alpha, prop_beta)
+     g_candidate_given_former = dbeta(candidate, prop_alpha, prop_beta)
+   } else if (prop_function == "uniform") {
+     # We can simplify for uniform, as it is symmetric: g(x|y) = g(y|x)
+     g_former_given_candidate = 1 / (d - c)
+     g_candidate_given_former = 1 / (d - c)
+   }
```

```
+
+   acc_probability = ((dens_candidate * g_former_given_candidate) /
+     (dens_former * g_candidate_given_former));
+
+   acc_probability = min(1, acc_probability)
+   return(acc_probability)
+ }
>
> # 1. (b)
>
> # Using c = 0.1, d = 0.9
> met_hastings_truncated_beta(100, 0.1, 0.9, "beta")
[1] "Interval: ( 0.1 , 0.9 )"
[1] "Proposal function: beta"
[1] "Acceptance probability: 0.75"
> met_hastings_truncated_beta(100, 0.1, 0.9, "uniform")
[1] "Interval: ( 0.1 , 0.9 )"
[1] "Proposal function: uniform"
[1] "Acceptance probability: 0.55"
>
> # Using c = 0.25, d = 0.75
> met_hastings_truncated_beta(100, 0.25, 0.75, "beta")
[1] "Interval: ( 0.25 , 0.75 )"
[1] "Proposal function: beta"
[1] "Acceptance probability: 0.39"
> met_hastings_truncated_beta(100, 0.25, 0.75, "uniform")
[1] "Interval: ( 0.25 , 0.75 )"
[1] "Proposal function: uniform"
[1] "Acceptance probability: 0.67"
```


Truncated Beta(2.7, 6.3) on (0.1 , 0.9) - beta
Frequency
0
1
2
3
4
5
6
0.1
0.2
0.3
0.4
0.5
0.6
0.7
x


Truncated Beta(2.7, 6.3) on (0.1 , 0.9) - uniform
Frequency
0
2
4
6
8
10
0.1
0.2
0.3
0.4
0.5
0.6
x





We notice the considerably higher acceptance rate for the beta function in the $(0.1, 0.9)$ range as compared to the uniform function (75% vs 55%). This makes sense – the range pretty much still continues to encapsulate most of the original pdf

for the beta, which is parameterized well and therefore is obviously is a better fit for the actual target beta function than a uniform function. However, this is flipped for the $(0.25, 0.75)$ range (39% vs 67%). This is because most of the range of the pdf that is relevant for the beta function gets cut out from the range.

2.

Exercise 6.9 Calculate the mean of a gamma $\mathcal{G}(4.3, 6.2)$ random variable using

a. Accept–Reject with a gamma $\mathcal{G}(4, 7)$ candidate;
b. Metropolis–Hastings with a gamma $\mathcal{G}(4, 7)$ candidate;
c. Metropolis–Hastings with a gamma $\mathcal{G}(5, 6)$ candidate.

In each case, monitor the convergence across iterations.

We need to calculate the mean of a gamma $G(4.3, 6.2)$ random variable. We know the true mean by definition to be $\frac{n}{\lambda} = \frac{4.3}{6.2} = 0.69354838709$.

(a)

For the accept-reject method with a gamma $G(4, 7)$ candidate, we need to first calculate constant M such that $M \geq sup_x[\frac{f(x)}{g(x)}]$:

$$\frac{f(x)}{g(x)} = \frac{G(4.3, 6.2)}{G(4, 7)}$$

which we can use code to try and investigate, and then proceed:

```
> x_seq <- seq(0.01, 20, length.out = 100)
>
> # Calculate ratio of f(x) / g(x) at each point
> ratios <- dgamma(x_seq, shape=4.3, rate=6.2) / dgamma(x_seq, shape=4,
rate=7)
> print(ratios)
  [1] 1.825208e-01 5.361883e-01 7.703168e-01 1.019992e+00 1.305276e+00
1.639118e+00
```

```
  [7] 2.033787e+00 2.502597e+00 3.060751e+00 3.725963e+00 4.519063e+00
5.464642e+00
 [13] 6.591792e+00 7.934962e+00 9.534960e+00 1.144013e+01 1.370777e+01
1.640574e+01
 [19] 1.961443e+01 2.342908e+01 2.796244e+01 3.334805e+01 3.974399e+01
4.733737e+01
 [25] 5.634968e+01 6.704297e+01 7.972734e+01 9.476962e+01 1.126037e+02
1.337429e+02
 [31] 1.587941e+02 1.884748e+02 2.236337e+02 2.652737e+02 3.145806e+02
3.729556e+02
 [37] 4.420550e+02 5.238358e+02 6.206104e+02 7.351110e+02 8.705652e+02
1.030786e+03
 [43] 1.220276e+03 1.444356e+03 1.709307e+03 2.022547e+03 2.392837e+03
2.830519e+03
 [49] 3.347805e+03 3.959112e+03 4.681459e+03 5.534937e+03 6.543258e+03
7.734414e+03
 [55] 9.141436e+03 1.080331e+04 1.276604e+04 1.508392e+04 1.782100e+04
2.105287e+04
 [61] 2.486871e+04 2.937372e+04 3.469203e+04 4.097006e+04 4.838055e+04
5.712723e+04
 [67] 6.745043e+04 7.963362e+04 9.401111e+04 1.109772e+05 1.309969e+05
1.546186e+05
 [73] 1.824890e+05 2.153706e+05 2.541627e+05 2.999255e+05 3.539092e+05
4.175878e+05
 [79] 4.926992e+05 5.812922e+05 6.857822e+05 8.090170e+05 9.543534e+05
1.125749e+06
 [85] 1.327868e+06 1.566209e+06 1.847254e+06 2.178642e+06 2.569378e+06
3.030074e+06
 [91] 3.573240e+06 4.213616e+06 4.968577e+06 5.858598e+06 6.907810e+06
8.144647e+06
```

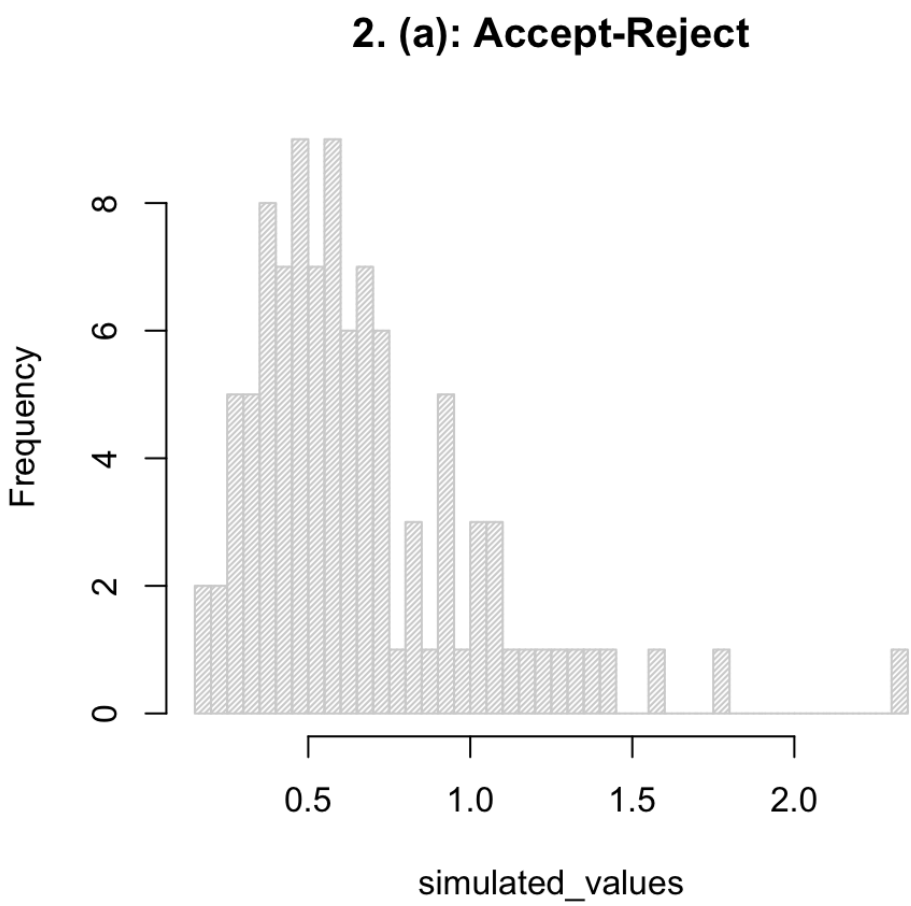
```
 [97] 9.602621e+06 1.132122e+07 1.334697e+07 1.573471e+07
>
> # The ratios seem to continue to infinity,
> # and took a long time to run when taking this M,
> # since the acceptance rate becomes extremely low.
> # Instead, I used G(4,6) with a heavier tail to better model
>
> ratios <- dgamma(x_seq, shape=4.3, rate=6.2) / dgamma(x_seq, shape=4,
rate=6)
> print(ratios)
  [1] 0.33477770 0.80365403 0.94347152 1.02085489 1.06752266 1.09545000
1.11069626
  [8] 1.11683343 1.11617694 1.11032784 1.10044584 1.08740073 1.07186240
1.05435774
 [15] 1.03530806 1.01505474 0.99387737 0.97200691 0.94963546 0.92692364
0.90400631
 [22] 0.88099703 0.85799163 0.83507098 0.81230340 0.78974647 0.76744862
0.74545045
 [29] 0.72378581 0.70248271 0.68156415 0.66104871 0.64095122 0.62128316
0.60205318
 [36] 0.58326740 0.56492975 0.54704229 0.52960539 0.51261803 0.49607790
0.47998165
 [43] 0.46432497 0.44910278 0.43430930 0.41993816 0.40598253 0.39243515
0.37928841
 [50] 0.36653446 0.35416519 0.34217236 0.33054757 0.31928235 0.30836817
0.29779649
 [57] 0.28755876 0.27764647 0.26805114 0.25876439 0.24977788 0.24108341
0.23267284
 [64] 0.22453819 0.21667159 0.20906530 0.20171173 0.19460342 0.18773308
0.18109356
```

```
 [71] 0.17467787 0.16847917 0.16249077 0.15670618 0.15111901 0.14572307
0.14051232
 [78] 0.13548087 0.13062298 0.12593307 0.12140573 0.11703566 0.11281774
0.10874699
 [85] 0.10481857 0.10102776 0.09737001 0.09384089 0.09043610 0.08715147
0.08398296
 [92] 0.08092666 0.07797875 0.07513557 0.07239354 0.06974922 0.06719926
0.06474041
 [99] 0.06236955 0.06008363
> M <- max(ratios)
> print(M)
[1] 1.116833
>
> accept_reject_gamma = function(n, M) {
+
+   simulated_values = rep(0, n)
+   simulated_successes = 0
+   simulated_trials = 0
+
+   # Target distribution: Gamma(4.3, 6.2)
+   target_shape = 4.3
+   target_rate = 6.2
+
+   # Candidate distribution: Gamma(4, 6) -- not Gamma(4, 7)
+   cand_shape = 4
+   cand_rate = 6
+
+   while (simulated_successes < n) {
+
+     simulated_trials = simulated_trials + 1
```

```
+
+     x_cand = rgamma(1, shape=cand_shape, rate=cand_rate)
+     uniform = runif(1, 0, M * dgamma(x_cand, shape=cand_shape, rate=cand_ra
+
+     # Accept if uniform < f(x_cand) where f is Gamma(4.3, 6.2)
+     if (uniform < dgamma(x_cand, shape=target_shape, rate=target_rate))
{
+       simulated_successes = simulated_successes + 1
+       simulated_values[simulated_successes] = x_cand
+     }
+
+     if (simulated_trials %% 10 == 0) {
+       print(paste("mean after", simulated_trials, "simulated_trials:",
mean(simulated_values)))
+     }
+   }
+
+   accept_prob = simulated_successes / simulated_trials
+   print(paste("Acceptance probability:", accept_prob))
+   print(paste("Estimated mean:", mean(simulated_values)))
+   hist(simulated_values, density = 50, breaks = 50, main = "2. (a): Accept-
+ }
>
> accept_reject_gamma(100, M)
[1] "mean after 10 simulated_trials: 0.057361156802528"
[1] "mean after 20 simulated_trials: 0.124639035681143"
[1] "mean after 30 simulated_trials: 0.174761781387247"
[1] "mean after 40 simulated_trials: 0.238516844987886"
[1] "mean after 50 simulated_trials: 0.301053141532097"
[1] "mean after 60 simulated_trials: 0.369637321321696"
```

```
[1] "mean after 70 simulated_trials: 0.446326576448048"
[1] "mean after 80 simulated_trials: 0.497404487331674"
[1] "mean after 90 simulated_trials: 0.534486438923785"
[1] "mean after 100 simulated_trials: 0.592475025527754"
[1] "mean after 110 simulated_trials: 0.653229654547747"
[1] "Acceptance probability: 0.900900900900901"
[1] "Estimated mean: 0.664852717498268"
```

With the new candidate gamma $G(4, 6)$, it converges very well. We can see the above means getting better over time.



(b)

Metropolis-Hastings with a gamma $G(4, 7)$ candidate:

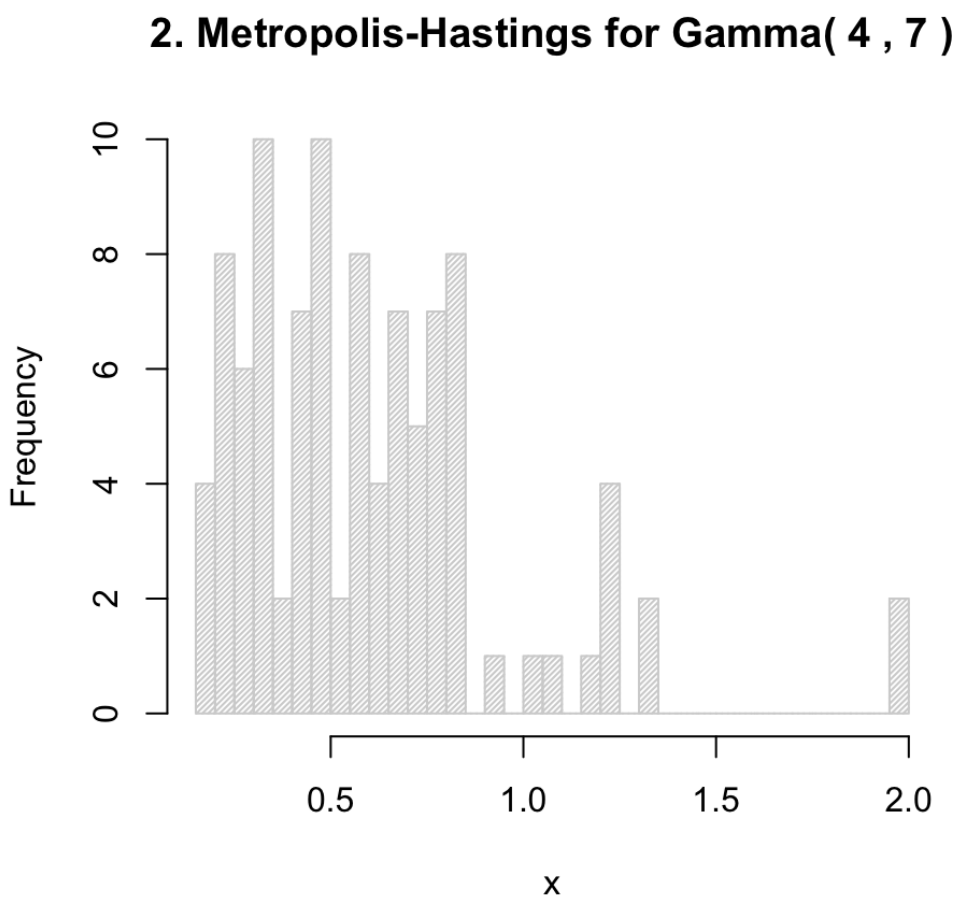
```
> met_hastings_gamma = function(n, prop_shape, prop_rate) {
```

```
+
+   x = rep(0, n)
+   j = 1
+   acc = 0
+   x[1] = 2
+
+   while (j < n) {
+
+     # Sample candidate value
+     x_cand = rgamma(1, shape = prop_shape, rate = prop_rate)
+
+     # Calculate acceptance probability
+     acc_prob = acc_prob_calc_gamma(x_cand, x[j], prop_shape, prop_rate)
+
+     # Decide whether to accept or not
+     acc_or_not = rbinom(1, 1, acc_prob);
+
+     # if accept or not
+     if (acc_or_not == 1) {
+       acc = acc + 1;
+       j = j + 1;
+       x[j] = x_cand
+     }
+     else {
+       j = j + 1
+       x[j] = x[j-1]
+     }
+
+     if (j %% 10 == 0) {
+       print(paste("mean after", j, "iterations:", mean(x[1:j])))
```

```
+     }
+   }
+
+   acc_probability = acc/n;
+   print(paste("Acceptance probability:", acc_probability));
+   print(paste("Estimated mean:", mean(mean(x[1:n]))))
+   hist(x, breaks = 50, density = 50, main = paste(
+     "2. Metropolis-Hastings for Gamma(", prop_shape, ",", prop_rate,
")")
+     );
+ }
>
> acc_prob_calc_gamma = function(candidate, former, prop_shape, prop_rate)
{
+
+   if (candidate < 0) {
+     acc_probability = 0;
+   }
+   else {
+     # Target densities
+     dens_candidate = dgamma(candidate, shape = 4.3, rate = 6.2)
+     dens_former = dgamma(former, shape = 4.3, rate = 6.2)
+
+     # Proposal densities
+     g_former_given_candidate = dgamma(former, shape = prop_shape, rate
= prop_rate)
+     g_candidate_given_former = dgamma(candidate, shape = prop_shape,
rate = prop_rate)
+
+     acc_probability = (dens_candidate * g_former_given_candidate) / (dens_
```

```
* g_candidate_given_former);
+   }
+
+   acc_probability = min(1, acc_probability);
+   return(acc_probability);
+ }
>
> # 2. (b): Using Gamma(4, 7)
> met_hastings_gamma(100, 4, 7)
[1] "mean after 10 iterations: 0.784790016282701"
[1] "mean after 20 iterations: 0.664775095539249"
[1] "mean after 30 iterations: 0.65066042996016"
[1] "mean after 40 iterations: 0.617259564762832"
[1] "mean after 50 iterations: 0.659536698996242"
[1] "mean after 60 iterations: 0.633522620303361"
[1] "mean after 70 iterations: 0.588662600598103"
[1] "mean after 80 iterations: 0.573466093970765"
[1] "mean after 90 iterations: 0.574118140801531"
[1] "mean after 100 iterations: 0.60192485306117"
[1] "Acceptance probability: 0.79"
[1] "Estimated mean: 0.60192485306117"
```

Using $G(4, 7)$, it converges well, but not as good. We can see the above means getting better over time, although it slightly underestimates.



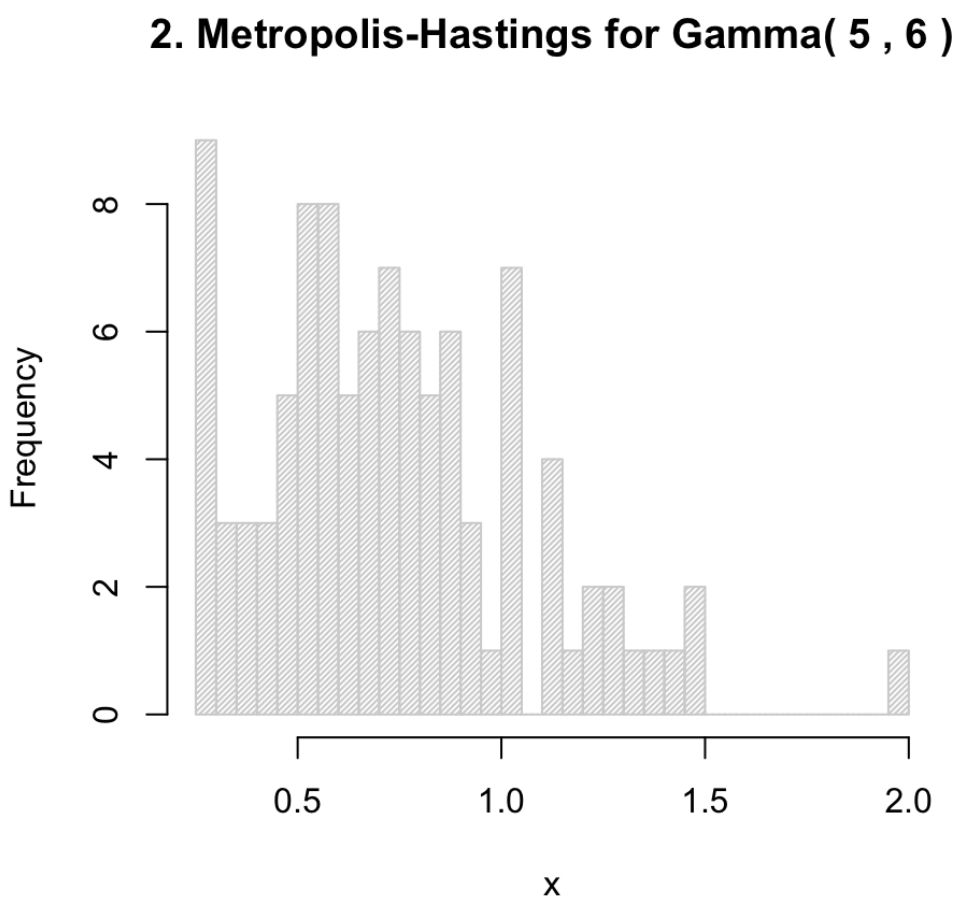
(c)

Metropolis-Hastings with a gamma $G(5, 6)$ candidate:

```
> # 2. (c): Using Gamma(5, 6)
> met_hastings_gamma(100, 5, 6)
[1] "mean after 10 iterations: 0.951377290724033"
[1] "mean after 20 iterations: 0.730853024169404"
[1] "mean after 30 iterations: 0.730709995413933"
[1] "mean after 40 iterations: 0.768977295580612"
[1] "mean after 50 iterations: 0.784397849443638"
[1] "mean after 60 iterations: 0.78316770471714"
[1] "mean after 70 iterations: 0.733693425666811"
[1] "mean after 80 iterations: 0.745312775744982"
```

```
[1] "mean after 90 iterations: 0.743769769713237"
[1] "mean after 100 iterations: 0.742317336251595"
[1] "Acceptance probability: 0.76"
[1] "Estimated mean: 0.742317336251595"
```

Using $G(4,7)$, it also converges very well. We can see the above means getting better over time, and it is slightly closer than using $G(4,7)$, although it slightly overestimates now.



3.

We show the calculations leading to the kernel expression:

$$K(X_t, X_{t+1}) = \rho(X_t, X_{t+1})\, g(X_{t+1}|X_t) + \delta(X_{t+1} - X_t)\,(1 - r(X_t)).$$

We know that

$$K(X_t, X_{t+1}) = K(X_t|X_{t+1})$$

To be thorough (and restating the reasoning behind what's given in the notes), we know by definition that the chain either moves to the candidate or doesn't change; i.e. our next state X_{t+1} is either:

- X_{cand} with probability $\rho(X_t, X_{cand})$
- X_t with probability $1 - \rho(X_t, X_{cand})$

which translates to the conditional probability density being

$$f(X_{t+1}|X_{cand}, X_t) = \delta(X_{cand} - X_{t+1})\rho(X_t, X_{cand}) + \delta(X_t - X_{t+1})(1 - \rho(X_t, X_{cand}))$$

as given. We then marginalize over all possible candidate values, i.e. taking

$$K(X_t, X_{t+1}) = \int_{\mathbb{R}} f(X_{t+1}|X_{cand} = x_{cand}, X_t) g(X_{cand} = x_{cand}|X_t) dx_{cand}$$

Plugging in:

$$= \int_{\mathbb{R}} [\delta(X_{cand} - X_{t+1})\rho(X_t, X_{cand}) + \delta(X_t - X_{t+1})(1 - \rho(X_t, X_{cand}))] g(x_{cand}|X_t) dx_{cand}$$

(simplifying a bit, for sanity)

$$= \int_{\mathbb{R}} [\delta(x - X_{t+1})\rho(X_t, x) + \delta(X_t - X_{t+1})(1 - \rho(X_t, x))] g(x|X_t) dx$$

$$= \int_{\mathbb{R}} \delta(x - X_{t+1})\rho(X_t, x) g(x|X_t) dx + \int_{\mathbb{R}} \delta(X_t - X_{t+1})(1 - \rho(X_t, x)) g(x|X_t) dx$$

Looking at the first integral, this essentially represents the sum of all the scenarios where we ended up at X_{t+1} – this is due to the product of the delta function representing that we reached the transition or not from the candidate value, the rho function representing the probability of accepting the candidate value, and the g function representing the probability of generating the candidate value. We therefore only care about the actual value of $x_{cand} = X_{t+1}$, since is the only place under the integral where the delta function is 1 (and nonzero). We can therefore simplify the first term accordingly based on this logic:

$$= \rho(X_t, X_{t+1}) g(X_{t+1}|X_t) + \int_{\mathbb{R}} \delta(X_t - X_{t+1})(1 - \rho(X_t, x)) g(x|X_t) dx$$

And continuing to integrate:

$$= \rho(X_t, X_{t+1})g(X_{t+1}|X_t) + \delta(X_t - X_{t+1}) \int_{\mathbb{R}} (1 - \rho(X_t, x))g(x|X_t)dx$$

$$= \rho(X_t, X_{t+1})g(X_{t+1}|X_t) + \delta(X_t - X_{t+1})[\int_{\mathbb{R}} g(x|X_t)dx - \int_{\mathbb{R}} \rho(X_t, x)g(x|X_t)dx]$$

$$= \rho(X_t, X_{t+1})g(X_{t+1}|X_t) + \delta(X_t - X_{t+1})[1 - \int_{\mathbb{R}} \rho(X_t, x)g(x|X_t)dx]$$

We now define $r(X_t) = \int_{\mathbb{R}} \rho(X_t, x)g(x|X_t)dx.$, which represents the combined probability of proposing and accepting some candidate value. We therefore now have

$$= \rho(X_t, X_{t+1})g(X_{t+1}|X_t) + \delta(X_t - X_{t+1})(1 - r(X_t))$$

and because the delta function is symmetric, we have

$$= \rho(X_t, X_{t+1})g(X_{t+1}|X_t) + \delta(X_{t+1} - X_t)(1 - r(X_t))$$

exactly as we sought to find.

4.

We are given the Baker suggested acceptance probability $p = \frac{f_X(x_{i+1})}{f_X(x_{i+1})+f_X(x_i)}$. We aim to find whether $g(x_{i+1}|x_i) = g(x_i|x_{i+1})$ guarantees detailed balance, which is defined in the notes to be $P(X = x_i)P(X_i \to X_{i+1}) = P(X = x_{i+1})P(X_{i+1} \to X_i)$. Let's evaluate this statement by solving either side. Starting with the left side:

$$P(X = x_i)P(X_i \to X_{i+1}) = f(x_i) \cdot g(x_{i+1}|x_i) \cdot \frac{f(x_{i+1})}{f(x_{i+1}) + f(x_i)}$$

$$P(X = x_i)P(X_i \to X_{i+1}) = \frac{f(x_i)f(x_{i+1})g(x_{i+1}|x_i)}{f(x_{i+1}) + f(x_i)}$$

Let's evaluate the right hand side and see where it goes:

$$P(X = x_{i+1})P(X_{i+1} \to X_i) = f(x_{i+1}) \cdot g(x_i|x_{i+1}) \cdot \frac{f(x_i)}{f(x_i) + f(x_{i+1})}$$

Given that $g(x_{i+1}|x_i) = g(x_i|x_{i+1})$:

$$= f(x_{i+1}) \cdot g(x_{i+1}|x_i) \cdot \frac{f(x_i)}{f(x_i) + f(x_{i+1})}$$

$$= \frac{f(x_i)f(x_{i+1})g(x_{i+1}|x_i)}{f(x_{i+1}) + f(x_i)}$$

This is the same as the left hand side, which indicates that this does indeed guarantee detailed symmetry. Intuitively, the probability works since it indicates the proportion of the total probability belongs to the x_{i+1} state, and the g function symmetry means that the proposed state generation doesn't favor either side. So there is equilibrium in the sense that only the acceptance probability matters, which is balanced out by the probability of being in the initial state, whether that is x_i or x_{i+1}.